

VIVA AND INTERVIEW GUIDE

PROJECT SENTINEL

Autonomous AI-Powered Payment Risk Intelligence & Protection Platform

Combining Machine Learning, Graph Intelligence, Agentic AI and PayPal Payment Infrastructure

PREPARED FOR

PayPal AI Hackathon 2026

PREPARED BY

Tanisha Dey

DATE

October 2026

Disclaimer: Sentinel is a hackathon prototype. It runs on PayPal Sandbox with synthetic data only and is not an official, deployed or endorsed PayPal fraud-prevention product. It does not access PayPal's internal risk systems. Model results describe synthetic data and are not evidence of real-world performance.

About this document

A plain-language primer on every technology, model and design choice, followed by 110 viva and interview questions with short answers, deeper explanations and follow-ups, and ready-made pitches and descriptions.

Item	Detail
Project	PROJECT SENTINEL: Autonomous AI-Powered Payment Risk Intelligence & Protection Platform
Prepared for	PayPal AI Hackathon 2026
Prepared by	Tanisha Dey
Source of truth	The repository (backend/, frontend/, docs/). Every metric and test result was taken from the running system or its test suite.
Repository	github.com/Tanisha-dey18/hackathon
Generated	2026-10-31

> **Disclaimer.** Sentinel is a hackathon prototype using PayPal Sandbox and synthetic data. It is not an officially deployed PayPal fraud prevention product and has no access to PayPal's internal systems. Features that are designed but not implemented are labelled **Future Scope**, and integrations that are implemented but were not executed live are labelled as such.

Contents

VIVA PRIMER: Sentinel in Plain Language . . . 4

Sentinel in simple words . . . 4
Every technology, and why it was chosen . . . 4
The ML models in one paragraph each . . . 4
Graph intelligence in one paragraph . . . 5
Agentic AI in one paragraph . . . 5
PayPal integration in one paragraph . . . 5
Backend, API and database in one paragraph . . . 5
Security in one paragraph . . . 5
Limitations to state honestly . . . 5

Part 19: Interview and Viva Preparation . . . 6

19.1 Non-technical questions (20) . . . 6
19.2 Technical questions (30) . . . 8
19.3 Machine learning questions (15) . . . 11
19.4 Backend and database questions (15) . . . 13
19.5 PayPal integration questions (15) . . . 15
19.6 AI agent and graph intelligence questions (15) . . . 16

Part 20: Personal Project Explanation . . . 19

20.1 30-second elevator pitch . . . 19
20.2 Two-minute explanation . . . 19
20.3 Five-minute technical explanation . . . 19
20.4 Resume description . . . 20
20.5 LinkedIn description . . . 20
20.6 GitHub repository description . . . 20
20.7 Hackathon submission summary . . . 20

SENTINEL

VIVA PRIMER: Sentinel in Plain Language

Sentinel in simple words

When you pay with PayPal online, two things happen: you **approve** the payment, and later the shop **captures** the money. Sentinel works in the gap between them. It checks whether the payment looks safe. Safe payments are captured straight away. Risky ones wait until a person has looked at the evidence and said yes. An AI assistant prepares that evidence, but it is never allowed to move money itself.

Every technology, and why it was chosen

Technology	What it is	Why Sentinel uses it
React + TypeScript	UI library + typed JavaScript	Two apps (shop and console) in one codebase; types catch API mistakes
Vite	Build tool	Fast dev server; proxies `/api` and `/ws` to the backend
Tailwind CSS	Utility CSS	Consistent dark design quickly
TanStack Query	Server-state cache	Caching and automatic refresh when WebSocket events arrive
Recharts	Charts	Dashboard, ROC/PR curves, SHAP bars
React Flow	Graph UI	Pan, zoom and click on the relationship graph
AG Grid Community	Data grid	Filter, sort and search 2,000+ transactions; CSV export
PayPal JS SDK	PayPal Buttons	Real Sandbox checkout without exposing the secret
FastAPI + Pydantic	Python web framework + validation	Typed endpoints, Swagger, WebSockets, same language as the ML code
SQLAlchemy + Alembic	ORM + migrations	One model layer for PostgreSQL and SQLite; versioned schema
PostgreSQL / SQLite	Databases	Postgres for deployment; SQLite for zero-setup local runs
Redis (optional)	In-memory store	Shared rate-limit counters across instances
scikit-learn, XGBoost	ML libraries	Standard, strong models for tabular fraud data
SHAP	Explainability	Explains each prediction feature by feature
NetworkX	Graph library	Finds shared devices, cycles and clusters
LangGraph	Agent workflow library	Clear, testable multi-step investigation
Groq	LLM API (optional)	Fast tool-calling LLM for the analysis step
bcrypt, PyJWT	Security libraries	Password hashing, signed login tokens
Docker, Render	Deployment	Reproducible environment; one-click cloud blueprint (configured, not deployed)

The ML models in one paragraph each

Logistic Regression adds up weighted features and turns the total into a probability. It is simple and transparent, but it only draws straight-line boundaries. **Random Forest** grows hundreds of decision trees on random samples and lets them vote. It is robust, and it is Sentinel's primary model because it had the best PR-AUC (0.897 on synthetic data). **XGBoost** builds trees one after another, each fixing the previous one's errors. It is often the strongest on tabular data; here it came third (0.886), and the project reports that honestly. **Isolation Forest** finds unusual points by how quickly random cuts can isolate them. It uses no labels and gives an "anomaly percentile". **SHAP** explains a single prediction by sharing out the model's output among the features.

Key ideas: **precision** (of what we flagged, how much was fraud), **recall** (of all fraud, how much we flagged), **PR-AUC** (the best single number when fraud is rare), **point-in-time features** (computed only from earlier transactions, so there is no peeking at the future), and **class weights** (making the rare fraud class count more).

Graph intelligence in one paragraph

Each payment links a customer to a device, a merchant and a destination wallet. Sentinel stores these links and builds a graph. A device used by three or more accounts is a "shared device". Money going A to B to C to D and back to A is a "cycle". Groups of accounts connected through devices and wallets are "clusters" (shops are left out, because everyone shops at the same places). Wallets on a synthetic watch list are "flagged destinations". Any of these can push a payment to HIGH risk and becomes evidence for the investigator.

Agentic AI in one paragraph

An agent is an AI that uses tools to look things up, instead of answering from memory. Sentinel's agent is a LangGraph workflow: retrieve the transaction, study the customer, read the ML prediction, check the graph, check the merchant, collect evidence, analyse, write the report, send it to a reviewer. Tools create numbered evidence (E1, E2 ...), and the report may only cite those numbers; anything else is deleted. If no LLM key is configured, a rule-based analysis is used and clearly labelled. The agent can recommend APPROVE, HOLD_FOR_REVIEW, REJECT or ESCALATE, but it cannot capture or reject a payment.

PayPal integration in one paragraph

The server gets an OAuth token with its secret, creates an order with Orders API v2 (`intent=CAPTURE`), and the buyer approves it in PayPal. On approval the server reads the order back and checks it is APPROVED, for the right amount and the right shopper. It captures only if the risk is LOW or a reviewer approved. A database lock and a fixed `PayPal-Request-Id` mean it can never capture twice. Webhooks are accepted only after PayPal's signature-verification API says they are genuine. PayPal's status and Sentinel's status are stored separately. Everything runs on the Sandbox; the live Sandbox run still needs to be performed with real credentials.

Backend, API and database in one paragraph

FastAPI routers (auth, transactions, intel, paypal, ops) call service modules (payments, risk, review, transactions, demo). Every request is validated by Pydantic and checked for role and ownership. Swagger is at `/docs`. One WebSocket pushes live events to the console. The database has 18 tables: core domain (customers, merchants, devices, destinations, products, transactions), ML (predictions, model versions), agent (investigations, evidence), graph (entities, relationships), payments (PayPal orders, payment events) and governance (reviewer decisions, audit logs, settings). Constraints protect integrity, for example amount > 0, a unique idempotency key and a unique capture ID.

Security in one paragraph

Secrets live only in environment variables. The browser never sees the PayPal secret. Passwords are hashed with bcrypt. Logins give a JWT with a role. Reviewers must re-enter their password and type the transaction ID to decide. Prices are calculated on the server. Every order checks ownership. Rate limits slow down abuse. The audit log is hash-chained, so editing any row is detectable.

Limitations to state honestly

Synthetic data only; scores are not calibrated; live PayPal Sandbox, webhooks and the Groq path not yet executed; Docker/Render configured but not run; single-process caches; no frontend tests, MFA or load tests.

PART 19

Interview and Viva Preparation

Each question has a **short answer** (say this first), a **deeper explanation** (use it if they want more), and sometimes a **likely follow-up**. Answers describe the implementation as it actually is, including what was not verified.

19.1 Non-technical questions (20)

Q1. What is Sentinel in one sentence?

Short answer: A layer that checks every PayPal checkout for fraud risk before the money is collected, and holds risky payments for a human.

Deeper explanation: It sits between the buyer approving in PayPal and the capture. Normal payments go through automatically; suspicious ones are explained, investigated by an AI agent and decided by a reviewer.

Follow-up: Why between approval and capture? Because that is the last moment you can stop money moving without refunding.

Q2. What problem are you solving?

Short answer: Stopping fraudulent payments before they are captured, without blocking genuine customers, and explaining every decision.

Deeper explanation: Single-payment scores miss organised fraud and give no reasons. Sentinel adds relationship analysis, explanations, an investigator and human control.

Q3. Who would use it?

Short answer: Merchants and their risk teams.

Deeper explanation: Shoppers use the storefront; reviewers and analysts use the console; admins tune the policy and retrain models.

Q4. Why does fraud matter to a merchant?

Short answer: Fraudulent sales usually come back as chargebacks, so the merchant loses the goods, the money and fees.

Deeper explanation: Blocking too much is also costly, because genuine customers leave. The goal is balance, which is why Sentinel holds rather than declines.

Q5. What does "held" mean for the customer?

Short answer: They approved the payment in PayPal, but no money has been taken yet.

Deeper explanation: The screen says the payment is under review and funds have not been captured. If approved, it is captured; if rejected, nothing is taken.

Q6. What makes Sentinel different from a normal fraud filter?

Short answer: It looks at relationships between accounts, explains itself and keeps a human in control of irreversible actions.

Deeper explanation: In our demo C the model scored a payment at about 0.31, but the graph showed a shared emulator device and mule wallets, and the system raised it to HIGH.

Q7. Where is the AI?

Short answer: In three places: the ML risk model, the anomaly detector and the investigation agent.

Deeper explanation: SHAP explains the model, and graph analysis provides network signals. The agent uses tools and writes an evidence-cited report.

Q8. Does the AI make the final decision?

Short answer: No. It only recommends.

Deeper explanation: The agent has no payment tools. Reviewers must re-enter their password and type the transaction ID to decide.

Q9. Is this used by PayPal?

Short answer: No. It is a hackathon prototype built on public PayPal Sandbox APIs and synthetic data.

Deeper explanation: It does not use or claim access to PayPal's internal fraud systems.

Q10. Is real customer data used?

Short answer: No. All customers, merchants, devices and history are generated.

Deeper explanation: The generator is deterministic and labels its own fraud patterns. That makes demos reproducible but means accuracy claims are limited to synthetic data.

Q11. What was the hardest part?

Short answer: Making the payment flow safe: no double captures, no bypassing review, and honest status.

Deeper explanation: That needed a compare-and-set lock, PayPal request IDs, ownership and amount checks, and separate PayPal and Sentinel status fields.

Q12. What does a reviewer see?

Short answer: The transaction, the model's reasons, a relationship graph and the AI report with evidence.

Deeper explanation: Plus an evidence timeline, the decision history and the audit trail, all in one workspace.

Q13. Why not just block suspicious payments automatically?

Short answer: Because false positives hurt genuine customers.

Deeper explanation: Holding gives a human the chance to verify. The policy is configurable if a business wants a stricter setup.

Q14. How do you know it works?

Short answer: 23 automated tests pass, and the models are evaluated on held-out data.

Deeper explanation: Hold-out PR-AUC of the primary model is 0.897 on synthetic data. Live PayPal Sandbox and Groq paths have not yet been run with real credentials.

Q15. What happens if PayPal is down?

Short answer: Sentinel returns a clear error with PayPal's debug ID and never pretends a payment succeeded.

Deeper explanation: If credentials are missing it returns 503 and the checkout explains the setup. Failed captures are marked `CAPTURE_FAILED` and can be retried safely.

Q16. How long does a risk check take?

Short answer: Each assessment's latency is measured and stored; the dashboard shows the average.

Deeper explanation: In development the average over the first live assessments was under a second, but it included cold-start model loading. No proper performance test was run.

Q17. What is the audit trail and why does it matter?

Short answer: A record of every action that is hash-chained, so changes are detectable.

Deeper explanation: Each entry includes the previous entry's hash. A verify button recomputes the chain; our test proves a modified row is detected.

Q18. What would a judge see in three minutes?

Short answer: A normal payment cleared, an unusual one held with explanations, and a fraud ring found by the graph, investigated and rejected.

Deeper explanation: With credentials, A and C run through real PayPal Sandbox checkout.

Q19. What did you learn?

Short answer: That safety in payments is mostly state management, and that AI is most useful when it is constrained.

Deeper explanation: Evidence validation and human-in-the-loop design mattered more than the choice of model.

Q20. What would you do next?

Short answer: Run live Sandbox and Groq tests, then validate on real fraud data.

Deeper explanation: Then calibration, monitoring, an AUTHORIZE-intent flow and scaling the event system.

19.2 Technical questions (30)

Q21. Describe the architecture.

Short answer: A React SPA, a FastAPI backend with risk, ML, graph and agent services, a relational database, and PayPal Sandbox.

Deeper explanation: REST plus one WebSocket; SQLAlchemy with Alembic; PostgreSQL in Docker/Render and SQLite locally; optional Redis for rate limiting; optional Groq for the LLM.

Q22. Why FastAPI?

Short answer: Typed validation with Pydantic, automatic Swagger and native WebSockets in Python.

Deeper explanation: Python lets the API, ML and graph code share one process and one codebase.

Q23. How does the frontend stay live?

Short answer: One authenticated WebSocket; events invalidate TanStack Query caches.

Deeper explanation: For example a `payment.updated` event refreshes orders, events and the transaction detail.

Q24. How do background threads push WebSocket messages?

Short answer: The server's event loop is saved at start-up and threads use `asyncio.run_coroutine_threadsafe`.

Deeper explanation: The agent and simulator run in threads, so they cannot await directly.

Q25. How is authentication done?

Short answer: bcrypt-hashed passwords and HS256 JWTs.

Deeper explanation: Role checks are FastAPI dependencies; the WebSocket validates the token from the query string and requires a staff role.

Q26. What roles exist?

Short answer: ADMIN, REVIEWER, ANALYST and CUSTOMER.

Deeper explanation: Analysts read and investigate, reviewers decide, admins change policy, retrain and decide escalations, customers shop.

Q27. How do you prevent a held payment from being captured?

Short answer: The capture function itself checks authorisation: LOW risk for automatic, or a stored APPROVE decision.

Deeper explanation: Even a direct call with `REVIEWER` authorisation is refused without a decision record. A test calls it directly and expects 409.

Q28. How do you prevent duplicate captures?

Short answer: A database compare-and-set lock plus a stable PayPal-Request-Id.

Deeper explanation: `UPDATE ... SET capture_state='IN_PROGRESS' WHERE capture_state IN ('NONE','FAILED')`; only the winner calls PayPal. `ORDER_ALREADY_CAPTURED` is reconciled by reading the order.

Q29. What is idempotency?

Short answer: Doing the same request twice has the same effect as doing it once.

Deeper explanation: Network retries and double clicks are normal, so payment APIs must be idempotent.

Q30. How are prices protected?

Short answer: The server recalculates prices from its catalogue; the client total is display-only.

Deeper explanation: On approval, PayPal's amount must match the stored amount, otherwise the order is marked `INTEGRITY_FAILED`.

Q31. Why can a cart only contain one seller?

Short answer: Each checkout creates one PayPal purchase unit and one transaction paying one merchant.

Deeper explanation: That keeps merchant risk and destination features meaningful. Multi-seller carts would need split payments.

Q32. How is the transaction ID generated?

Short answer: Prefix plus a base-36 timestamp plus 3 random characters, for example `TXN_56SIN7VDWC`.

Deeper explanation: Synthetic history uses sequential IDs like `TXN_100542`.

Q33. How does the risk policy work?

Short answer: Thresholds map probability to a level; escalation rules can only raise it.

Deeper explanation: LOW < 0.30 <= MEDIUM < 0.70 <= HIGH < 0.90 <= CRITICAL. Anomaly >= 0.98 or amount >= 10x average gives at least MEDIUM; graph signals give at least HIGH.

Q34. Can the policy be changed?

Short answer: Yes, by an ADMIN in Settings; changes are validated and audited.

Deeper explanation: Thresholds must be strictly increasing; old and new values are stored in the audit log.

Q35. What is stored for each prediction?

Short answer: Probability, anomaly and graph scores, level, action, top factors, SHAP values, features, policy notes and latency.

Deeper explanation: This lets you reconstruct exactly why a decision was made, even after a model is retrained.

Q36. How do you handle model versions?

Short answer: Each training run creates a version with one row per algorithm; one is marked primary.

Deeper explanation: Artifacts are stored as joblib bundles; old versions remain. If the active artifact is missing at start-up, the backend retrains.

Q37. How does the audit hash chain work?

Short answer: Each row's hash is SHA-256 over its content and the previous hash.

Deeper explanation: Verification reads rows in order (bypassing cached objects) and recomputes. A modified row breaks the chain at that row.

Q38. Why store PayPal events separately?

Short answer: To keep a raw record of every PayPal response and webhook.

Deeper explanation: It includes HTTP status and `paypal-debug-id`, which PayPal support asks for.

Q39. How is rate limiting implemented?

Short answer: Fixed windows in Redis if configured, otherwise a sliding window in memory.

Deeper explanation: Login 10/min, create-order and capture 20/min, webhook 300/min, general default configurable.

Q40. How do you handle CORS?

Short answer: An explicit allow-list from `CORS_ORIGINS`, limited methods and headers.

Deeper explanation: Allowed headers are Authorization, Content-Type and Idempotency-Key.

Q41. How are secrets kept out of logs?

Short answer: A logging filter redacts bearer tokens, client secrets and access tokens.

Deeper explanation: The PayPal secret is only used for OAuth with HTTP basic auth.

Q42. How is the system deployed?

Short answer: Docker Compose locally, or a Render Blueprint with managed PostgreSQL.

Deeper explanation: The frontend is built and served by nginx (Compose) or as a Render static site. These configurations were written but not deployed during development.

Q43. What happens on first start?

Short answer: Migrations run, the synthetic world is generated, models are trained and history is scored.

Deeper explanation: About 30 to 60 seconds on the development laptop.

Q44. How are migrations handled?

Short answer: Alembic, run programmatically at start-up.

Deeper explanation: One initial migration creates 18 tables; offline SQL generation for PostgreSQL was checked.

Q45. How did you test PayPal without credentials?

Short answer: With an httpx `MockTransport` that emulates PayPal Sandbox endpoints, in tests only.

Deeper explanation: It implements OAuth, order create/get/capture with request-ID idempotency, already-captured errors and webhook verification. The live Sandbox path still needs a real run.

Q46. How does the graph explorer layout work?

Short answer: Kamada-Kawai layout per connected component, computed on the server.

Deeper explanation: Coordinates are returned with the nodes; React Flow renders them and fits the view after nodes are measured.

Q47. Why AG Grid?

Short answer: Filtering, sorting, quick search, pagination and CSV export over thousands of rows.

Deeper explanation: We used the Community edition; row grouping is Enterprise, so we used risk filter chips.

Q48. What would break first at scale?

Short answer: The in-process feature state, graph cache and WebSocket fan-out.

Deeper explanation: They assume one process. You would need a shared feature store, a graph store and pub/sub.

Q49. How would you scale to millions of transactions?

Short answer: Stream ingestion, a shared online feature store, a model-serving tier, incremental graph updates in a graph database, and a message bus for events.

Deeper explanation: Keep the capture lock in the database, partition by customer, cache graph signals per entity, and run investigations asynchronously from a queue.

Q50. What is the biggest technical debt?

Short answer: Single-process caches and untested live integrations.

Deeper explanation: Plus no frontend tests and uncalibrated probabilities.

19.3 Machine learning questions (15)

Q51. Why did you choose XGBoost?

Short answer: It is a strong default for tabular data, but Sentinel does not force it: the primary model is whichever has the best PR-AUC.

Deeper explanation: In v1 Random Forest won (0.897) over Logistic Regression (0.893) and XGBoost (0.886). We report that honestly.

Follow-up: Why might XGBoost lose here? The data is small (1,500 training rows), and bagging is more stable on small, noisy data.

Q52. Why is accuracy insufficient for fraud detection?

Short answer: With 7% fraud, predicting "never fraud" is 93% accurate and useless.

Deeper explanation: Precision, recall and PR-AUC focus on the rare class.

Q53. What is the difference between precision and recall?

Short answer: Precision is how many flagged payments are really fraud; recall is how much fraud we flag.

Deeper explanation: Raising the threshold usually increases precision and lowers recall. The primary model had precision 0.941 and recall 0.889 at 0.5.

Q54. Why PR-AUC over ROC-AUC for selection?

Short answer: PR-AUC reflects performance on the rare class; ROC-AUC can look good because there are so many true negatives.

Deeper explanation: Logistic Regression had the best ROC-AUC (0.952) but much lower precision (0.744).

Q55. How did you handle class imbalance?

Short answer: Class weights for LR and RF, `scale_pos_weight` for XGBoost, and stratified splitting.

Deeper explanation: No oversampling such as SMOTE was used, which avoids creating synthetic minority points.

Q56. What is data leakage and how did you avoid it?

Short answer: Using information that would not be available at prediction time.

Deeper explanation: Features use only earlier transactions; imputation is fitted on training data; labels are never features; the destination "flagged" status is external intelligence, not derived from labels.

Q57. Explain point-in-time features.

Short answer: Each transaction's features are computed from history before its timestamp.

Deeper explanation: The same `FeatureState` class is used in training (replay) and serving (state from the database), so definitions match.

Q58. How do you handle missing values?

Short answer: Keep them as NaN and impute the training median inside each pipeline.

Deeper explanation: First-time customers have no average, previous location or interval. `is_first_txn` makes that explicit.

Q59. What is Isolation Forest used for?

Short answer: An unsupervised anomaly percentile used by the policy and shown to reviewers.

Deeper explanation: It is trained without labels; its PR-AUC (0.830) is reported for transparency.

Q60. What is SHAP?

Short answer: A method that splits a prediction into per-feature contributions based on Shapley values.

Deeper explanation: TreeExplainer for tree models, LinearExplainer for LR; positive contributions become reviewer-readable sentences.

Follow-up: What space are the values in? Probability for Random Forest; log-odds for XGBoost and LR.

Q61. What are the most important features?

Short answer: Amount, log amount, hour of day, distance from home and night-time.

Deeper explanation: Hour and night rank high because the synthetic ring operates at night, a generator artefact we disclose.

Q62. Are your probabilities calibrated?

Short answer: No; they are risk scores.

Deeper explanation: Class weighting shifts them. Isotonic or Platt calibration is future work.

Q63. What happens if the model wrongly flags a genuine customer?

Short answer: The payment is held, not declined, and a reviewer can approve it, which captures the payment.

Deeper explanation: That approval is recorded and becomes a "legitimate" label for the next retraining.

Q64. How would you evaluate on real data?

Short answer: Out-of-time validation, cost-weighted metrics and monitoring after launch.

Deeper explanation: Train on earlier months, test on later months, and choose thresholds by expected loss rather than F1.

Q65. How is the model retrained?

Short answer: ADMIN triggers `POST /api/models/retrain`; a new version is created.

Deeper explanation: Labels come from synthetic ground truth and reviewer decisions on live payments.

19.4 Backend and database questions (15)

Q66. Why SQLAlchemy and not raw SQL?

Short answer: One model layer for PostgreSQL and SQLite with type safety.

Deeper explanation: It still allows explicit atomic updates like the capture lock.

Q67. Why both PostgreSQL and SQLite?

Short answer: PostgreSQL for deployment, SQLite for zero-setup local runs.

Deeper explanation: Render provides `postgres://` URLs, which the config converts for the psycopg 3 driver.

Q68. What constraints protect data integrity?

Short answer: Foreign keys, check constraints and unique keys.

Deeper explanation: Examples: amount > 0, allowed statuses, allowed roles, unique PayPal order ID, idempotency key, capture ID and webhook event ID.

Q69. Why is `transaction_id` unique in `paypal_orders`?

Short answer: One Sentinel transaction maps to exactly one PayPal order.

Deeper explanation: It prevents two orders paying for the same transaction.

Q70. How do you store JSON?

Short answer: SQLAlchemy JSON columns.

Deeper explanation: Used for metrics, SHAP values, features, policy notes, investigation steps, reports and raw PayPal payloads.

Q71. How are timestamps handled?

Short answer: Timezone-aware UTC.

Deeper explanation: SQLite drops timezone info, so an `aware()` helper re-attaches UTC.

Q72. What indexes exist?

Short answer: On foreign keys, statuses, risk levels and composite customer+time and device+time.

Deeper explanation: The composite indexes support history queries for features and tools.

Q73. How are errors returned?

Short answer: Consistent JSON with appropriate HTTP status codes.

Deeper explanation: 422 has a structured `errors` list; 502 for PayPal errors includes the debug ID; unhandled errors return a generic 500.

Q74. How are long-running tasks handled?

Short answer: Background threads for investigations, the simulator and retraining.

Deeper explanation: Each uses its own database session. A queue (Celery, RQ) would be the production choice.

Q75. How do you avoid two investigations of the same transaction?

Short answer: If one is RUNNING, its ID is returned instead of starting another.

Deeper explanation: The test for demo C relies on this deduplication.

Q76. How does `GET /api/transactions/{id}` build the case?

Short answer: It joins the transaction, related entities, predictions, investigations, decisions, PayPal order, events and audit rows.

Deeper explanation: That single call powers the workspace's left and bottom panels.

Q77. How is the latest prediction per transaction selected?

Short answer: A subquery on `max(id)` grouped by transaction.

Deeper explanation: Used by lists, the review queue and the graph engine.

Q78. What happens if a decision call fails halfway?

Short answer: Decision and audit are committed before the capture, so the record exists even if PayPal fails.

Deeper explanation: A failed capture sets `CAPTURE_FAILED`; the lock moves to FAILED and allows a safe retry with the same PayPal-Request-Id.

Q79. Why is the webhook handler run in a thread pool?

Short answer: It reads the raw body asynchronously, then runs blocking database and HTTP code in a worker thread.

Deeper explanation: This keeps the event loop free.

Q80. What found a bug late in development?

Short answer: Copying `.env.example` to `.env` made `DATABASE_URL` blank and broke start-up.

Deeper explanation: Fixed with `env_ignore_empty=True` in the settings, so blank values fall back to defaults.

19.5 PayPal integration questions (15)

Q81. How does PayPal Sandbox work?

Short answer: It is PayPal's test environment with test business and personal accounts and the same REST APIs, but no real money.

Deeper explanation: You create a Sandbox REST app for a client ID and secret, and approve payments with a Sandbox personal account.

Q82. What is the difference between order creation and payment capture?

Short answer: Creation defines what will be paid; capture actually moves the money after the buyer approves.

Deeper explanation: Created, then Approved (buyer), then Completed (capture). Sentinel's risk check sits between approval and capture.

Q83. Which PayPal APIs do you call?

Short answer: OAuth token, Orders v2 create, get and capture, and webhook signature verification.

Deeper explanation: `/v1/oauth2/token`, `/v2/checkout/orders`, `/v2/checkout/orders/{id}`, `/v2/checkout/orders/{id}/capture`, `/v1/notifications/verify-webhook-signature`.

Q84. Where does the PayPal secret live?

Short answer: Only in the backend environment.

Deeper explanation: The browser gets only the public client ID from `/api/paypal/config`.

Q85. Why intent CAPTURE and not AUTHORIZE?

Short answer: It is simpler and the approval-then-capture gap is enough to gate risk.

Deeper explanation: AUTHORIZE would hold funds longer and allow an explicit void on rejection. It is listed as future scope.

Q86. What does PayPal-Request-Id do?

Short answer: It makes a PayPal POST idempotent: repeating it returns the original result.

Deeper explanation: Sentinel uses the client's Idempotency-Key for creation and `capture-{order_id}` for capture.

Q87. How do you verify the order before capture?

Short answer: GET the order from PayPal and check status, amount, currency, reference and ownership.

Deeper explanation: Status must be APPROVED; a mismatch returns 409 and nothing is captured.

Q88. How are webhooks verified?

Short answer: By calling PayPal's verify-webhook-signature API with the transmission headers and webhook ID.

Deeper explanation: Only `verification_status = SUCCESS` is accepted; event IDs are de-duplicated.

Q89. Which webhook events are handled?

Short answer: Order approved and capture completed, denied, pending, refunded and reversed.

Deeper explanation: They update `paypal_status` and `capture_status`, and are stored in `payment_events`.

Q90. What if the buyer cancels in PayPal?

Short answer: No approval happens, so nothing is assessed or captured.

Deeper explanation: The checkout shows "Checkout cancelled in PayPal. Nothing was charged."

Q91. What if capture returns PENDING?

Short answer: Sentinel records PENDING and does not mark the transaction captured.

Deeper explanation: A later `PAYMENT.CAPTURE.COMPLETED` webhook would complete it.

Q92. Did you test against live Sandbox?

Short answer: Not yet: the credentials were not available on the build machine.

Deeper explanation: The flow is fully tested against a mock transport. Running demos A and C with real Sandbox credentials is the next verification step.

Q93. Do you use PayPal's fraud tools?

Short answer: No. Sentinel is a merchant-side layer using public APIs.

Deeper explanation: It complements, and does not replace or access, PayPal's own risk systems.

Q94. How would you add refunds?

Short answer: Call the Captures refund API for a captured payment and handle refund webhooks.

Deeper explanation: Refund status would be stored as a PayPal-sourced field, with a reviewer action and an audit entry.

Q95. Why is PayPal central rather than optional?

Short answer: The risk decision only matters because it controls a real capture.

Deeper explanation: PayPal's separation of approval and capture, request IDs and signed webhooks are part of the safety design.

19.6 AI agent and graph intelligence questions (15)

Q96. What is LangGraph?

Short answer: A library for building LLM workflows as state machines with nodes and edges.

Deeper explanation: Sentinel's graph has nine nodes, from retrieving the transaction to routing the report to a reviewer, with a typed shared state.

Q97. How does your AI agent use tools?

Short answer: Core tools always run in fixed steps; in LLM mode Groq can call extra tools before submitting a structured report.

Deeper explanation: Tool results become evidence keys, which the report must cite.

Q98. What happens if Groq is unavailable?

Short answer: A deterministic rule-based report is produced and clearly labelled as not LLM-generated.

Deeper explanation: The reason (not configured, or the error) is stored in the report and shown in the UI.

Q99. How do you prevent AI hallucinations?

Short answer: Evidence comes from tools, not the LLM, and the report validator removes any finding that does not cite a valid evidence key.

Deeper explanation: Invalid recommendations are normalised; if nothing valid remains, the rule-based report is used. Tests check that invented keys are removed.

Q100. What is a structured output here?

Short answer: The `submit_investigation_report` tool call with a fixed schema.

Deeper explanation: risk_level, summary, findings with evidence_keys, recommendation from four allowed values, and rationale.

Q101. Why can't the agent capture or reject payments?

Short answer: Those are irreversible financial actions and LLMs can be wrong.

Deeper explanation: The agent has no payment tools; humans decide with step-up authentication.

Q102. What evidence categories exist?

Short answer: ML score, anomaly, SHAP drivers, amount, device, geography, impossible travel, card testing, velocity, shared device, flagged destination, circular payments, cluster and merchant risk.

Deeper explanation: Each has a severity, and some are marked mitigating, such as a known device or a normal amount.

Q103. How does your graph detect suspicious relationships?

Short answer: Shared devices (3 or more accounts), money loops via `simple_cycles`, flagged-destination payments and connected clusters.

Deeper explanation: Merchant hubs are excluded from clustering so popular shops do not link everyone.

Q104. Why exclude merchants from clusters?

Short answer: Almost every customer pays popular merchants, which would merge everyone into one component.

Deeper explanation: Devices and wallets are much stronger signals of common control.

Q105. Did you implement community detection?

Short answer: No, connected components only.

Deeper explanation: Louvain or Leiden would find sub-communities inside large components; it is future scope.

Q106. How is a circular payment found?

Short answer: Build a customer-to-customer graph from wallet transfers and search for cycles of length 3 to 6.

Deeper explanation: The synthetic loop CUS_0081 to 0082 to 0083 to 0080 moved $16,064.37.

Q107. How does the graph affect the risk level?

Short answer: Graph signals raise risk to at least HIGH.

Deeper explanation: Triggers are a payment to a flagged destination, a device shared by 3 or more accounts, or a cycle member. A graph risk score is also stored.

Q108. How is the graph kept current?

Short answer: Each assessed transaction is written to the graph tables and the cache is invalidated.

Deeper explanation: The engine rebuilds on next use; this is fine for thousands of rows but would need incremental updates at scale.

Q109. What would a graph neural network add?

Short answer: Learned patterns over neighbourhoods instead of hand-written rules.

Deeper explanation: It needs more and real labelled data; it is future scope.

Q110. What would you improve with six more months?

Short answer: Live PayPal and Groq validation, real data, calibration, monitoring and scale.

Deeper explanation: Also AUTHORIZE flow, community detection, step-up customer verification, frontend tests and a security review.

PART 20

Personal Project Explanation

20.1 30-second elevator pitch

"I built Sentinel for the PayPal AI Hackathon. It protects PayPal checkouts by checking each payment after the buyer approves it but before the money is captured. It combines a machine-learning risk model, a graph that spots fraud rings through shared devices and mule wallets, and an AI investigator that writes an evidence-backed report. Normal payments go straight through; risky ones are held until a reviewer approves them. Everything is explained and audited."

20.2 Two-minute explanation

"Most fraud checks give you a score for one payment. Two things are missing: organised fraud looks normal one payment at a time, and a score without reasons still leaves someone to investigate.

Sentinel is a payment layer on top of PayPal. Our demo shop creates a real PayPal Sandbox order on the server. After the buyer approves, and before we capture, we fetch the order from PayPal, check the amount and that it belongs to the shopper, and score it. The score uses 26 features computed only from earlier transactions, from a Random Forest that was picked automatically because it had the best PR-AUC on held-out data, plus an Isolation Forest for anomalies and SHAP to explain the result.

Then the graph. We store every relationship between customers, devices, wallets and merchants, and NetworkX finds shared devices, money loops and clusters. In our third demo the model only gave about 0.31, but the device was shared with three other accounts paying flagged wallets, so the policy raised it to HIGH.

High-risk payments get an AI investigation built with LangGraph: seven tools collect evidence, and every finding has to cite an evidence item or it is removed. The agent can only recommend. A reviewer re-enters their password and approves, rejects or escalates, and only an approval lets PayPal capture. There are 23 automated tests, including ones that prove a high-risk payment can't be captured without approval and nothing is captured twice."

20.3 Five-minute technical explanation

"Let me walk through the stack and the guarantees.

Frontend: React and TypeScript with Vite and Tailwind. TanStack Query handles data, and one WebSocket invalidates queries so the console is live. React Flow draws the graph, Recharts the charts, AG Grid the transaction table, and the PayPal React SDK renders the buttons with only the public client ID.

Backend: FastAPI with Pydantic validation, SQLAlchemy and Alembic, PostgreSQL for deployment and SQLite locally. JWT auth with four roles.

Payment lifecycle: create-order recalculates prices on the server, creates a transaction and calls Orders v2 with intent CAPTURE, using the client's Idempotency-Key as PayPal-Request-Id. On capture-order we GET the order, require APPROVED, match amount, currency, reference and owner, then run risk. PayPal status is only written from PayPal responses; Sentinel status is separate. Capture uses a compare-and-set update so only one request proceeds, plus a stable PayPal-Request-Id and reconciliation if PayPal says it was already captured. The capture function re-checks authorisation: LOW risk, or a stored APPROVE decision.

ML: a deterministic synthetic world with labelled fraud patterns plus deliberate noise. Point-in-time features via one FeatureState class for training and serving. Stratified 75/25 split, class weights, four models, metrics on the hold-out only. Random Forest won with PR-AUC 0.897; XGBoost was third. SHAP gives per-payment explanations.

Policy: thresholds map to LOW, MEDIUM, HIGH or CRITICAL; escalations for anomaly, 10x amount and graph signals only raise risk.

Graph: relationships persisted in tables, loaded into NetworkX, transaction nodes collapsed into an entity projection. Shared devices, simple_cycles for loops, connected components excluding merchant hubs for clusters.

Agent: a nine-node LangGraph workflow. Evidence is generated deterministically from tool outputs; Groq can call more tools and must submit a structured report; a validator drops uncited findings; if Groq isn't available there's a labelled rule-based fallback.

Audit and tests: a SHA-256 hash-chained audit log with verification. 23 pytest tests with a mock PayPal transport. What I haven't verified yet is live Sandbox and live Groq, because I didn't have credentials on the build machine, and the models are only validated on synthetic data."

20.4 Resume description

Sentinel: AI payment risk platform on PayPal (PayPal AI Hackathon 2026). Built a full-stack payment protection layer (FastAPI, React/TypeScript) that risk-gates PayPal Orders v2 captures. Implemented point-in-time feature engineering and compared Random Forest, XGBoost, Logistic Regression and Isolation Forest (primary selected by hold-out PR-AUC 0.897 on synthetic data) with SHAP explanations; NetworkX graph detection of shared-device rings and circular payments; a LangGraph investigation agent with evidence-validated reports; idempotent, bypass-proof capture; and a hash-chained audit log. 23 automated tests.

20.5 LinkedIn description

I built Sentinel for the PayPal AI Hackathon 2026: a payment protection layer that checks every PayPal checkout between buyer approval and capture. It combines an ML risk model with SHAP explanations, graph analysis that finds fraud rings through shared devices and mule wallets, and a LangGraph AI investigator whose every claim must cite real evidence. Risky payments are held until a reviewer approves, and nothing can be captured twice. Built with FastAPI, React, scikit-learn, XGBoost, NetworkX, LangGraph and the PayPal Orders API on Sandbox with synthetic data.

20.6 GitHub repository description

Sentinel: AI-powered payment risk intelligence on PayPal Sandbox. ML + SHAP, NetworkX graph intelligence, LangGraph investigation agent, human-in-the-loop capture gating, hash-chained audit. FastAPI + React. Synthetic data.

20.7 Hackathon submission summary

Sentinel is a payment protection layer for PayPal checkouts. Between buyer approval and capture, it verifies the PayPal order, scores risk with an ML model (Random Forest selected by hold-out PR-AUC, explained with SHAP) and an anomaly detector, and checks a relationship graph for shared devices, mule wallets and money loops. LOW-risk payments are captured automatically; others are held, investigated by a LangGraph agent whose findings must cite tool-produced evidence, and decided by a re-authenticated human reviewer. Captures are idempotent and cannot bypass review, PayPal status and Sentinel status are kept separate, and every action is recorded in a hash-chained audit log. Built with FastAPI, React, scikit-learn, XGBoost, SHAP, NetworkX, LangGraph/Groq and PayPal Orders v2 on Sandbox, using synthetic data only.